# YAWM AI

A 10-Agent Hybrid Daily Planner for Ramadan

LangGraph · LangChain · ChromaDB · MCP Servers · Custom Python Tools

Hala Lakhal | March 1, 2026

## Motivation

Ramadan is a blessed month — but also a deeply demanding one. As Muslims we are called to observe five daily prayers, recite the Quran, perform morning and evening adhkar, fast from Suhoor to Iftar, and continue every worldly responsibility: work, study, family, personal growth. Managing all of this manually leads to overwhelm, missed intentions, and poor time use.

**YAWM AI** was built to solve this. It is a fully autonomous production-grade multi-agent system that reads real tasks from Notion, retrieves personal preferences from a RAG database (ChromaDB), respects Ramadan-specific constraints (prayer times, sleep cycles, Laylat Al-Qadr), validates the schedule for Salah conflicts, and writes a complete conflict-free day to Google Calendar — with a visual PNG card and a curated Islamic podcast notification sent by email and WhatsApp.

## Agent Overview

YAWM AI is composed of **10 specialized agents** (including Agent 0 Router and Agent 7.5 ConflictChecker) orchestrated as a hybrid directed graph using LangGraph. Each agent has a single responsibility and communicates exclusively through a shared typed state (`YawmState`).

### Version 1 — Pure Graph Pattern

The first version of YAWM AI used a **pure Graph Agent Pattern**: every user message, regardless of intent, entered the same fixed 9-node DAG and triggered the full planning pipeline. Asking *"what adhkar should I do?"* or *"when is Fajr?"* would run all agents sequentially — TaskCollector, Planner, Supervisor, the parallel trio, DayPlanner, CanvaAgent, DeenPodcast, and CalendarAgent — producing a 30-second response when a 2-second answer was sufficient. Every interaction was a full plan, whether the user needed one or not.

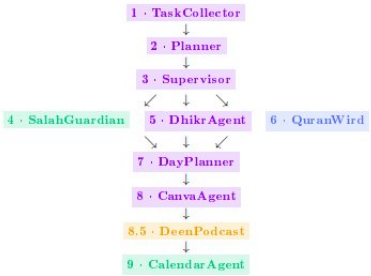



Figure 1: Version 1 — Pure Graph Pattern: every message enters the full 9-node pipeline. No routing, no bypass.

### Version 2 — Hybrid Pattern with Agents as Tools

The evolved architecture introduces **Agent 0: the Router** — an intent classifier (temperature=0) that sits at the entry point of the graph. It reads the user's message and makes a single decision:

- **full_plan** → the complete pipeline runs (Agents 1–9)
- **direct_query** → the Router calls the right specialist agent directly as a tool — Quran, Dhikr, Salah, Sleep, or General — and returns an instant answer. The full pipeline is never triggered.

This is the **Agents as Tools** pattern: specialist agents are registered as callable tools that the Router can invoke directly, bypassing all unnecessary computation. A direct Islamic question gets a 2-second response instead of a 30-second one. The full pipeline only runs when the user explicitly requests a daily plan.

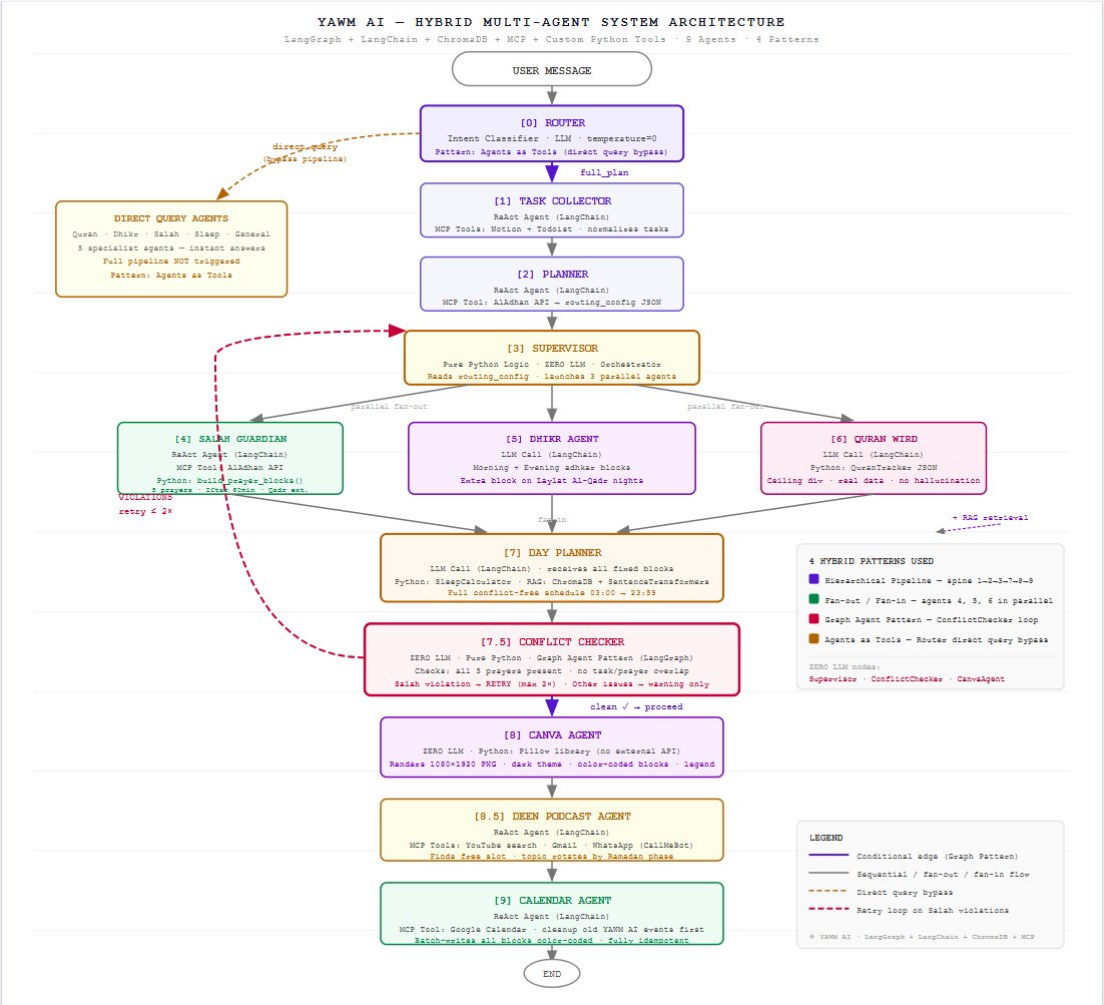



Figure 2: Version 2 — Hybrid architecture: Router bypass (Agents as Tools), hierarchical spine (1–3), parallel fan-out (4–6), ConflictChecker retry loop (7.5).

## Pattern Choice and Justification — Hybrid Pattern

The **first version** used a **pure Graph Agent Pattern**: every user message entered the same 9-node DAG regardless of intent — efficient for planning, but wasteful for simple queries. The **evolved architecture** adopts a **Hybrid Pattern** combining four distinct patterns, each chosen for a specific problem:

| Pattern | Where used | Why |
|---|---|---|
| **Agents as Tools** | Router (Agent 0) | Bypasses the full pipeline for direct queries (dhikr, salah, sleep). Instant response, no wasted compute. |
| **Hierarchical Pipeline** | Agents 1–3 (spine) | Sequential execution: task collection → planning → supervision. Each stage feeds the next with validated data. |
| **Fan-out / Fan-in** | Agents 4, 5, 6 (parallel) | SalahGuardian, DhikrAgent, QuranWird are fully independent. LangGraph runs them concurrently, saving ∼30 seconds per run. |
| **Graph Agent (conditional)** | Agent 7.5 ConflictChecker | Creates a bounded retry loop (max 2). Salah violations trigger replanning; warnings pass through silently. |

Both LangChain and LangGraph are used throughout. **LangChain** provides `ChatOpenAI`, `create_react_agent`, and the `langchain-mcp-adapters` bridge that wraps MCP server tools into LangChain-compatible tools automatically. **LangGraph** provides `StateGraph` for the graph topology, native parallel fan-out, conditional edges for the retry loop, and `astream` for live streaming to the web interface.

## What Each Agent Does

| Agent | Responsibility |
|---|---|
| **0 · Router** | Intent classifier (temperature=0). Decides whether the user needs a `full_plan` (triggers pipeline) or a `direct_query` (routes instantly to a specialist agent: Quran, Dhikr, Salah, Sleep, or General). Implements the Agents-as-Tools pattern. |
| **1 · TaskCollector** | ReAct agent. Calls Notion MCP and Todoist MCP to fetch all pending tasks. Normalises them into a unified list with id, title, priority, deadline, and estimated minutes. Google Calendar is never read — write-only. |
| **2 · Planner** | ReAct agent. Calls AlAdhan MCP to get today's prayer times and analyses the user's mood, Ramadan day, and task list. Produces a `routing_config` JSON with energy level and deep-work window. |
| **3 · Supervisor** | Pure Python — ZERO LLM. Reads `routing_config` and launches the three parallel agents (4, 5, 6). Acts as the fan-out gate and quality validator before scheduling. |
| **4 · SalahGuardian** | ReAct agent. Calls AlAdhan MCP for real prayer times. Builds five fixed prayer blocks with correct durations: Iftar extended to 60 min at Maghrib, Isha extended to 120 min on Laylat Al-Qadr nights (21, 23, 25, 27, 29). Updates prayer streak in local storage. |
| **5 · DhikrAgent** | LLM call (LangChain). Generates morning adhkar block (post-Fajr) and evening adhkar block (post-Asr). Adds an extra dhikr block on Laylat Al-Qadr nights. No MCP tools needed. |
| **6 · QuranWird** | LLM call (LangChain). Reads the persistent `quran_progress.json` tracker. Calculates today's page count via ceiling division based on Ramadan day. Returns `quran_blocks` and khatm percentage. Real data — no hallucination. |
| **7 · DayPlanner** | Core scheduling intelligence. Receives all fixed blocks (prayers, dhikr, Quran, sleep/Suhoor, Qailulah nap), flexible Notion tasks, and the sleep window from `SleepCalculator` (real 90-min cycle engine). Queries ChromaDB via SentenceTransformers to retrieve top-8 personal preferences (RAG). Uses LLM to build a complete conflict-free schedule 03:00–23:59. |
| **7.5 · ConflictChecker** | Pure Python — ZERO LLM. Graph Agent Pattern with conditional edge. Checks: all 5 prayers present, no task/prayer overlap. Salah violations trigger a retry to Supervisor (max 2). Other issues become warnings only — an imperfect schedule is always better than an infinite loop. |
| **8 · CanvaAgent** | Pure Python — ZERO LLM. Renders the `full_schedule` into a 1080×1920 PNG using Pillow. Each block is color-coded by type with a legend. Saves to `output/daily_schedule.png`. |
| **8.5 · DeenPodcast** | ReAct agent. Searches YouTube for an English Islamic video matching the Ramadan day theme and mood. Finds the best free 35-min slot. Sends a formatted Gmail notification and optional WhatsApp alert via CallMeBot. |
| **9 · CalendarAgent** | ReAct agent. First deletes all today's events tagged `Created by YAWM AI` (clean slate). Then batch-writes the complete `full_schedule` to Google Calendar with correct colors and idempotent descriptions. |

## MCP Servers

All external integrations run as **MCP (Model Context Protocol) servers** — independent subprocesses communicating over stdio JSON-RPC. Agents never call APIs directly; they call MCP tools via `langchain-mcp-adapters`, which wraps them into standard LangChain tools automatically.

| MCP Server | External API | Tools exposed |
|---|---|---|
| `notion_mcp.py` | Notion API v1 | `notion_list_tasks`, `notion_complete_task` |
| `google_calendar_mcp.py` | Google Calendar API v3 | `gcal_list_events`, `gcal_create_event`, `gcal_delete_event` |
| `aladhan_mcp.py` | AlAdhan REST API (free) | `get_prayer_times`, `get_hijri_date` |
| `todoist_mcp.py` | Todoist REST API v2 | `todoist_list_tasks`, `todoist_complete_task` |
| `deen_notify_mcp.py` | YouTube API + Gmail + CallMeBot | `search_deen_youtube`, `send_gmail_notify`, `send_whatsapp_notify` |
| `canva_mcp.py` | Local Pillow renderer | `render_schedule_card` — generates PNG from schedule blocks |

**RAG layer (ChromaDB + SentenceTransformers):** Personal user preferences are stored as vector embeddings using `all-MiniLM-L6-v2`. DayPlanner queries the store at runtime to retrieve top-8 semantically relevant preferences (e.g. hair = 2h, coding = 2–3h, shower in the evening) and injects them into the scheduling prompt.

**Custom Python tools (non-LLM):** `SleepCalculator` fits complete 90-min cycles backward from 03:15; `QuranTracker` reads/writes persistent JSON progress; `build_prayer_blocks()` constructs fixed prayer blocks deterministically; `retrieve_preferences()` queries ChromaDB by semantic similarity.

## System Test

**Tester:** Hala Lakhal **Date:** 22 February 2026 **Ramadan Day:** 5 **Mood:** Energized

**Input prompt:**

*"Energized, need to finish my presentation, plan my day properly"*

**Pipeline result:**

- 18 schedule blocks built across 10 block types
- ConflictChecker: 2 violations found on first pass (hair overlapping Maghrib) → retry → resolved
- 17 events written to Google Calendar after cleanup of old YAWM events
- Prayer streak: 18 days · Quran: Day 5 · Juz 16 complete · 55.5% khatm
- Deen Podcast: *RAMADAN 2026 IS APPROACHING* by Mufti Menk — scheduled 20:30
- Gmail + WhatsApp notification sent automatically
- RAG preferences respected: hair = 2h block (16:20–18:20), shower moved to evening, coding = 2h block

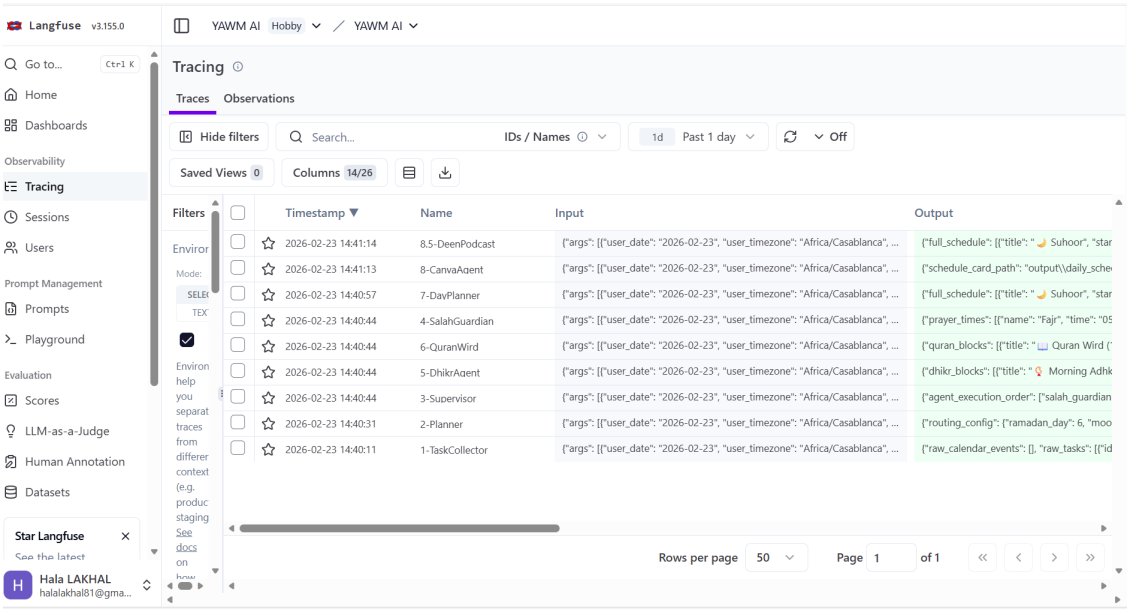
## ConflictChecker Output

```
[ConflictChecker] ❌ 2 violation(s) found (retry 0/2):
  → OVERLAP: 'Get hair done' ends 18:30 but '🌅 Maghrib' starts 18:27
  → PRAYER_BLOCKED: 'Get hair done' overlaps with 🌅 Maghrib

[ConflictChecker] ✅ Schedule validated — 17 blocks, 3 warning(s)
  ⚠ WARNING_OVERLAP: 'Quran Wird (2 pages)' ends 07:04 but 'Sunrise' starts 06:59
  ⚠ WARNING: 'Morning shower + bathroom routine' at 07:19 — ideally scheduled after 18:00
  ⚠ WARNING: No Deen podcast block found
```

Figure 4: ConflictChecker detecting 2 Salah violations on retry 0/2, then validating the corrected schedule with 3 warnings.

## LangFuse Tracing



## Web Interface — Chat & Live Agent Logs 2

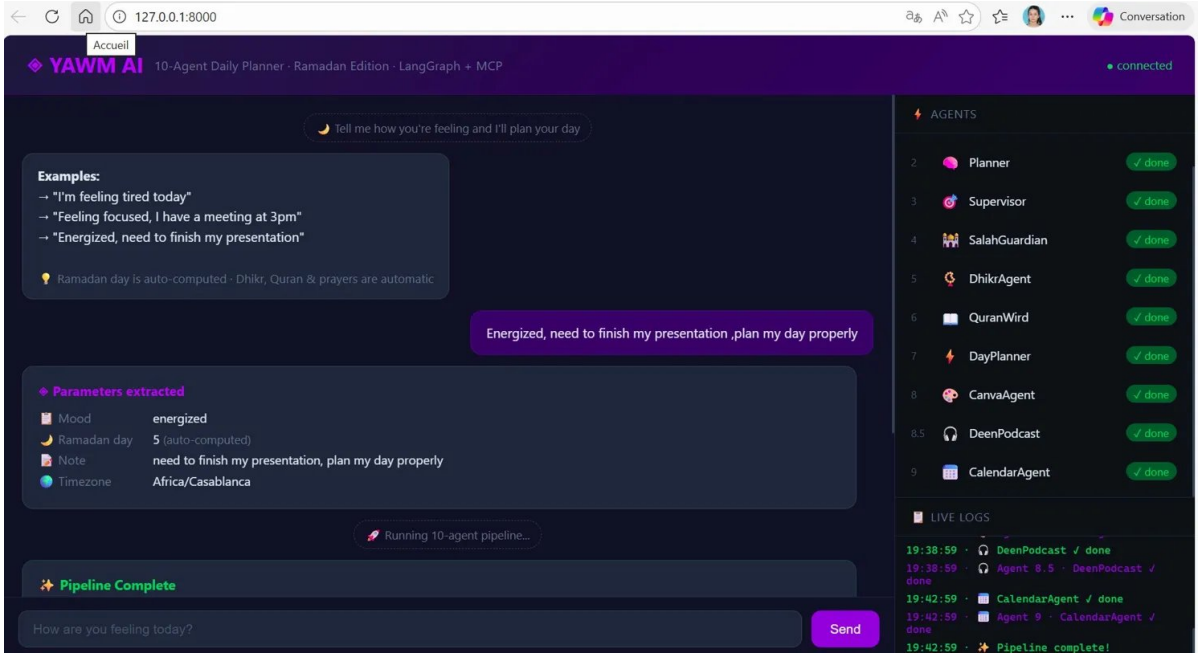



Figure 5: YAWM AI web interface: natural language input, extracted parameters, all 10 agents completing in real time with live logs.

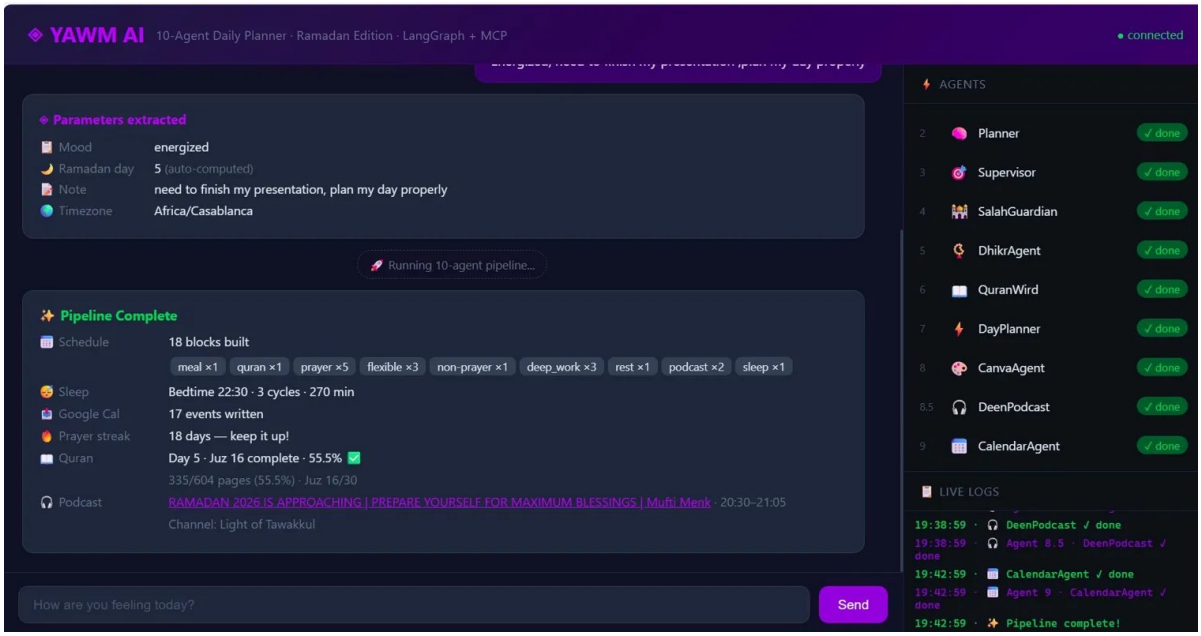



Figure 6: Pipeline Complete summary: 18 blocks, sleep window, Google Calendar events written, prayer streak, Quran progress, podcast link.

## Google Calendar Output

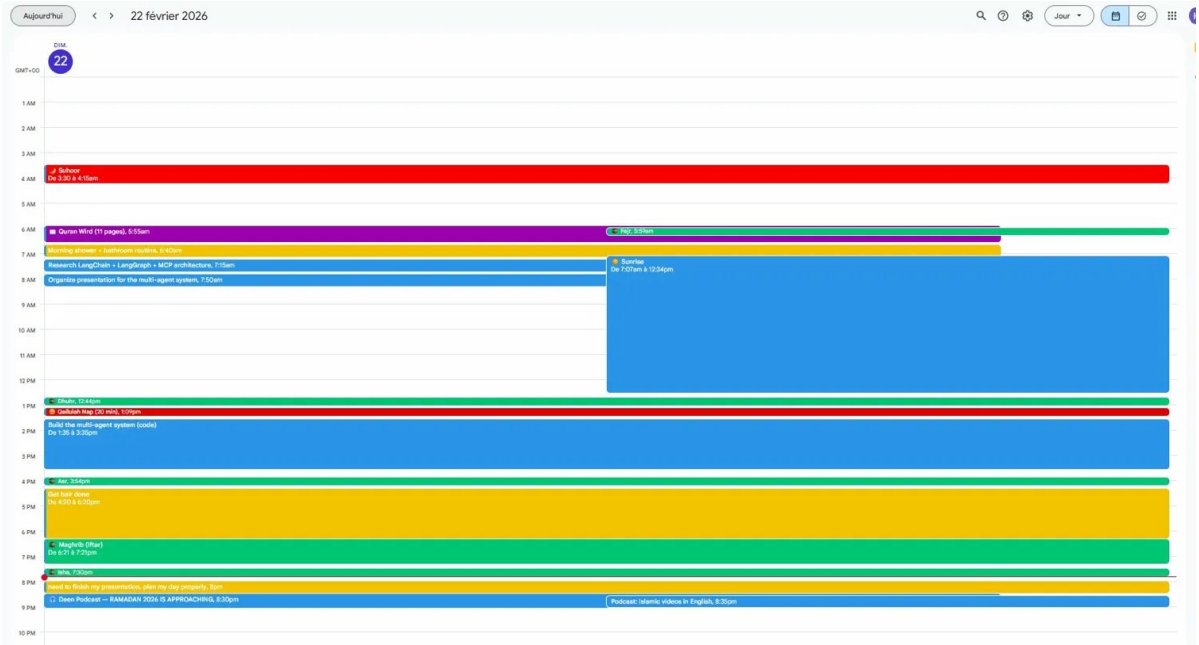



Figure 7: Google Calendar for 22 February 2026: Suhoor, Fajr, Quran Wird, deep work (coding + presentation), prayers, Get hair done (2h), Maghrib/Iftar, Isha, Deen Podcast, sleep — all color-coded and conflict-free.

## Gmail Notification

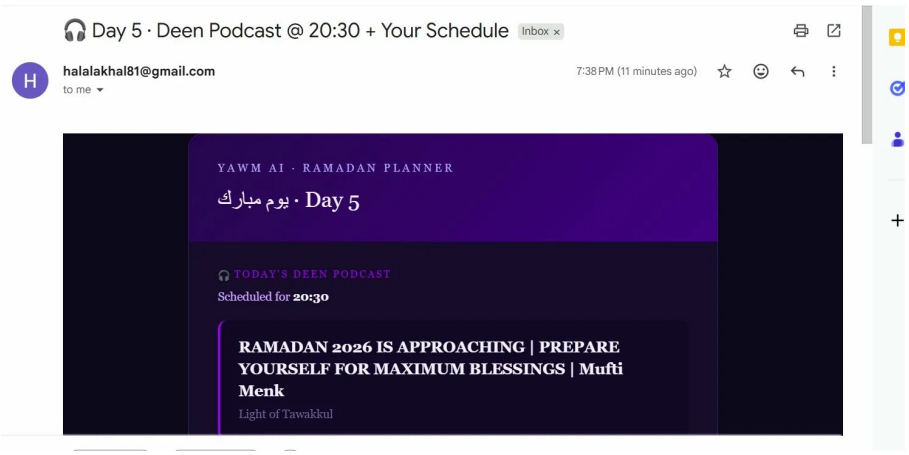


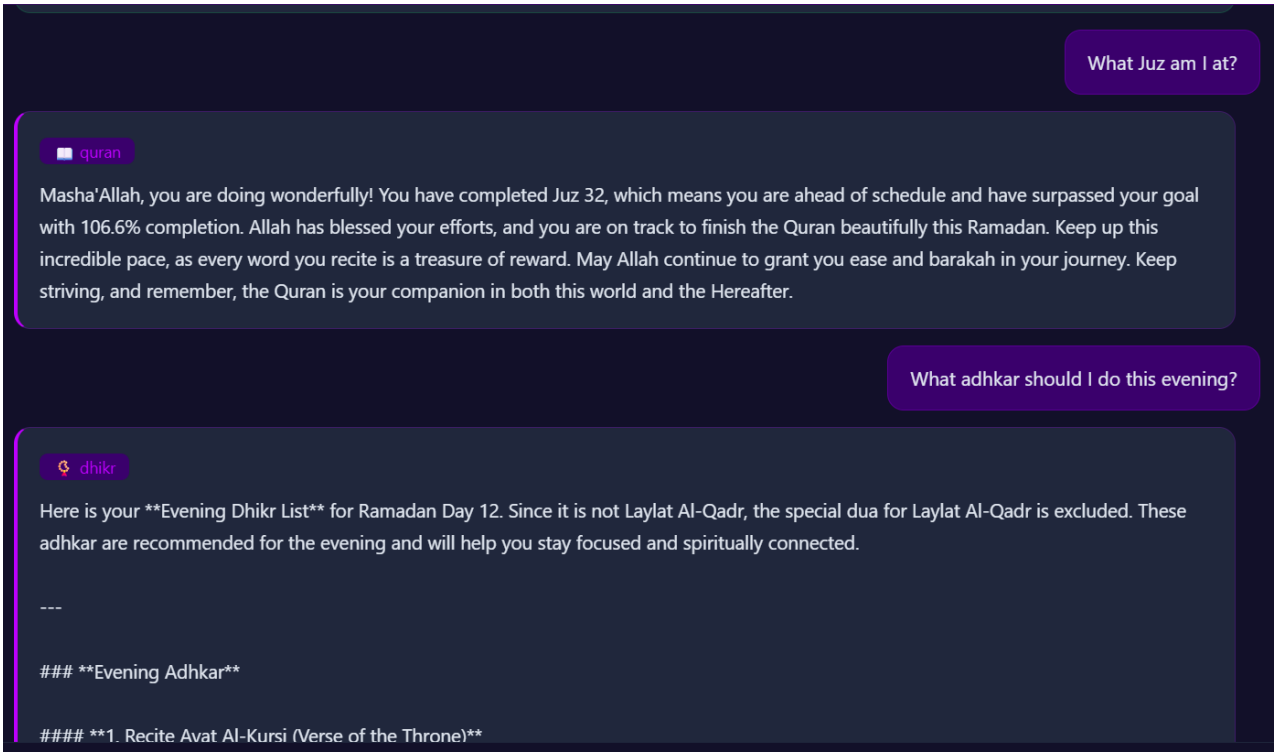

Figure 8: Automated Gmail notification: Ramadan Day 5, podcast scheduled at 20:30, Watch on YouTube button, schedule card embedded.

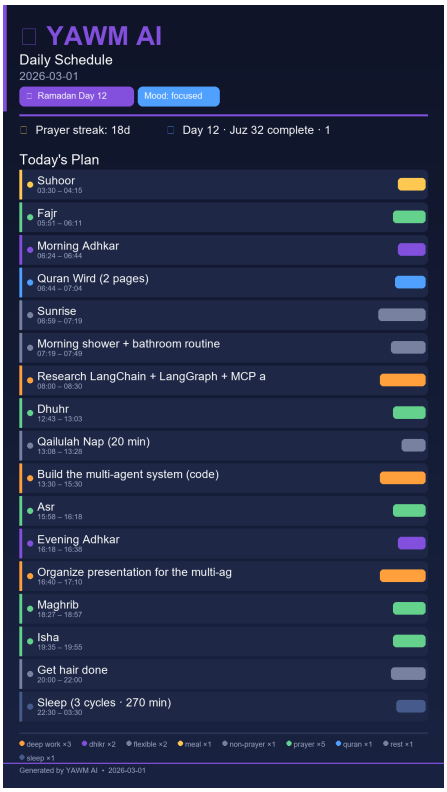
## Conclusion

YAWM AI demonstrates that a production hybrid multi-agent system built with LangChain, LangGraph, ChromaDB, and MCP can fully automate the Ramadan daily planning loop — from intent classification to calendar writing, with zero manual steps. The evolution from a pure Graph Pattern to a Hybrid Pattern added efficiency (Router bypass), resilience (ConflictChecker retry), and

precision (RAG-enriched scheduling). Every component is real: real APIs, real LLM calls, real notifications, real Google Calendar events. The system is extensible — adding a new agent, a new MCP server, or a new RAG preference requires no changes to the existing pipeline.

## Testing each subagent_Agents as Tools Pattern




# OutPut